

ChordSymbol ホワイトリスト統合パッケージ (music21準拠・自動整形+検証)

このキャンパスには、あなたの **コードシンボル・リストファイル** を中核にした、 1) **ホワイトリスト (music21準拠)** の生成、 2) **YAML chordmapの自動整形・検証** を行うための最小ツール一式をまとめています。

目的：autoChordは“土台”、仕上げは“人の彩色”。出力の**正確性（表記）**は機械で担保する。

📁 構成

- `chord_symbol_whitelist.yaml` ... ルート×サフィックス（品質/拡張）で **生成可能な集合** を定義するテンプレ
- `build_whitelist_from_list.py` ... あなたの**生のコード一覧（TXT/CSV）** からホワイトリストYAMLを生成（music21で検証しつつ）
- `validate_chordmap_plus.py` ... chordmap.yaml を **自動正規化→music21検証→ホワイトリスト照合**（未許可は警告/エラー）

既存の `validate_chordmap.py` は単独でも使えます。本パッケージは**ホワイトリスト**を追加導入した上位版です。

1) `chord_symbol_whitelist.yaml`（テンプレート）

ルートはA-G と `#`（シャープ）/`-`（フラット）を許容。サフィックスは **正規表現**でも**列挙**でも可。

```
# chord_symbol_whitelist.yaml
version: 1
roots: [A, B, C, D, E, F, G, A#, C#, D#, F#, G#, A-, B-, D-, E-, G-, A-, C-]
# 註: music21ではルートのフラットは "-" を使う (Bb→B-)。

suffix:
# === コア（autoChordの上限） ===
triads:
  allow: ["", m, o, +] # maj(無印)/min/dim/aug
sevenths:
  allow: [7, maj7, m7, m7b5, o7]

# === 表現拡張（人が付与） ===
sus_add:
  allow: [sus2, sus4, 6, add9, 6add9]
tensions_light:
  allow: [9, maj9, 7add9, 7#11, 7b13]

# === Altered（altは不可。個別列挙のみ） ===
altered:
```

```
allow: ["7b9#9#11b13", "7b9#11", "7#9#11", "7b9b13"]
```

オプション: 完全列挙のホワイトリスト (roots×suffixから自動生成に加え、個別追加)

explicit:

- N.C.

policy:

unknown: error # error|warn|ignore (ホワイトリスト外の表記に対する挙動)

2) build_whitelist_from_list.py

あなたの既存リスト (例: `my_chords.txt`) を読み、
- 正規化 (Bb→B-, E7-9→E7b9、Am(add9)→Amadd9、△→maj、ø7→m7b5 など) - music21 で検証 - 重複排除→YAML 生成 を行います。

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
build_whitelist_from_list.py
Usage:
python build_whitelist_from_list.py my_chords.txt --out chord_symbol_whitelist.yaml
python build_whitelist_from_list.py my_chords.csv --col 0 --out chord_symbol_whitelist.yaml
"""

from __future__ import annotations
import argparse, csv, re
from pathlib import Path
from typing import List, Set

# --- normalization helpers (music21準拠) ---
UNICODE_FLAT = re.compile(r"[ ♭ ]")
UNICODE_SHARP = re.compile(r"[ # ]")
TRIANGLE = re.compile(r"[ △ Δ ]")
PARENS = re.compile(r"[ ( ) ]")
COMMA = re.compile(r",")
ROOT_FLAT = re.compile(r"^(^/)([A-G])b")
TENSION_HYPHEN = re.compile(r"-(?=\d)")
HALF_DIM = re.compile(r"^[A-G](?:[#-]|b)?(.*)(ø7(.*)$)")
ALT = re.compile(r"alt(?:![a-zA-Z])")

ALT_EXPANSION = "b9#9#11b13"

def normalize(s: str) -> str:
    x = s.strip()
    if not x: return x
    x = UNICODE_FLAT.sub("b", x)
    x = UNICODE_SHARP.sub("#", x)
    x = TRIANGLE.sub("maj", x)
    if " " in x: x = x.replace(" ", "")
    x = COMMA.sub("", x)
```

```

x = PARENS.sub("", x)
m = HALF_DIM.match(x)
if m: x = f"{m.group(1)}{m.group(2)}m7b5{m.group(3)}"
x = re.sub(r"(?![a-zA-Z])sus(?:[24])", "sus4", x)
x = ALT.sub(ALT_EXPANSION, x)
x = ROOT_FLAT.sub(lambda m: f"{m.group(1)}{m.group(2)}-", x)
x = TENSION_HYPHEN.sub("b", x)
return x

# --- music21 verify ---
def verify_m21(ch: str) -> bool:
    try:
        from music21 import harmony
        harmony.ChordSymbol(ch)
        return True
    except Exception:
        return False

def read_list(path: Path, col: int | None) -> List[str]:
    if path.suffix.lower() in {".csv", ".tsv"}:
        sep = "," if path.suffix.lower() == ".csv" else "\t"
        out: List[str] = []
        with path.open("r", encoding="utf-8") as f:
            for row in csv.reader(f, delimiter=sep):
                if not row: continue
                s = row[col or 0]
                out.append(s)
        return out
    else:
        return [line.rstrip("\n") for line in path.read_text(encoding="utf-8").splitlines()]

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("listfile", type=str)
    ap.add_argument("--col", type=int, default=None, help="CSV/TSVの列番号")
    ap.add_argument("--out", type=str, default="chord_symbol_whitelist.yaml")
    args = ap.parse_args()

    src = Path(args.listfile)
    items = [normalize(s) for s in read_list(src, args.col)]
    uniq: Set[str] = {s for s in items if s}

    ok, ng = [], []
    for s in sorted(uniq):
        (ok if verify_m21(s) else ng).append(s)

    import yaml
    data = {
        "version": 1,

```

```

    "roots":
["A","B","C","D","E","F","G","A#","C#","D#","F#","G#","A-","B-","D-","E-","G-","C-"],
    "suffix": {
        "triads": {"allow": [ "", "m", "+", "o" ]},
        "sevenths": {"allow": [ "7", "maj7", "m7", "m7b5", "o7" ]},
        "sus_add": {"allow": [ "sus2", "sus4", "6", "add9", "6add9" ]},
        "tensions_light": {"allow": [ "9", "maj9", "7add9", "7#11", "7b13" ]},
        "altered": {"allow": [ "7b9#9#11b13", "7b9#11", "7#9#11", "7b9b13" ]},
    },
    "explicit": ok, # あなたのリストをそのまま追加
    "policy": {"unknown": "warn"}
}
Path(args.out).write_text(yaml.safe_dump(data, allow_unicode=True, sort_keys=False),
encoding="utf-8")
print(f"wrote {args.out} (ok={len(ok)}, ng={len(ng)})")
if ng:
    print("NG (music21で未受理):")
    for s in ng:
        print(" ", s)

if __name__ == "__main__":
    main()

```

3) validate_chordmap_plus.py

- YAMLを再帰走査 → 正規化 → music21で検証 → ホワイトリスト照合
- 既存の validate_chordmap.py より、ホワイトリスト基準のチェックが追加されています。

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
validate_chordmap_plus.py
Usage:
python validate_chordmap_plus.py chordmap.yaml
--whitelist chord_symbol_whitelist.yaml
--autofix --out fixed_chordmap.yaml
--verify-with-music21 --report out/report.json
"""
from __future__ import annotations
import argparse, json, re
from pathlib import Path
from typing import Any, Dict, Tuple, Union, List

try:
    import yaml
except Exception:
    yaml = None

```

```

# --- normalization (同等ロジック) ---
UNICODE_FLAT = re.compile(r"[ ♭ ]")
UNICODE_SHARP = re.compile(r"[ # ]")
TRIANGLE = re.compile(r"[△△]")
PARENS = re.compile(r"[\(\)]")
COMMA = re.compile(r",")
ROOT_FLAT = re.compile(r"^(^/)([A-G])b")
TENSION_HYPHEN = re.compile(r"-(?=\d)")
HALF_DIM = re.compile(r"^[A-G](?:[#-]|b)?(.*)(\d7)(.*)$")
ALT = re.compile(r"alt(?:[a-zA-Z])")
SUS_BARE = re.compile(r"(?![a-zA-Z])sus(?:[24])")
ALT_EXPANSION = "b9#9#11b13"

CODE_LIKE = re.compile(r"^\s*[A-G](?:[#b-])?[a-zA-Z0-9/+]*\s*$")

def normalize(s: str) -> str:
    x = s.strip()
    if not x: return x
    x = UNICODE_FLAT.sub("b", x)
    x = UNICODE_SHARP.sub("#", x)
    x = TRIANGLE.sub("maj", x)
    x = x.replace(" ", "") if " " in x else x
    x = COMMA.sub("", x)
    x = PARENS.sub("", x)
    m = HALF_DIM.match(x)
    if m: x = f"{m.group(1)}{m.group(2)}m7b5{m.group(3)}"
    x = SUS_BARE.sub("sus4", x)
    x = ALT.sub(ALT_EXPANSION, x)
    x = ROOT_FLAT.sub(lambda m: f"{m.group(1)}{m.group(2)}-", x)
    x = TENSION_HYPHEN.sub("b", x)
    return x

def verify_m21(ch: str) -> Tuple[bool, str | None]:
    try:
        from music21 import harmony
        harmony.ChordSymbol(ch)
        return True, None
    except Exception as e:
        return False, f"{type(e).__name__}: {e}"

def flatten_strings(obj: Any, path: Tuple[Union[str,int],...]=()) -> List[Tuple[Tuple[Union[str,int],...],
str]]:
    out = []
    if isinstance(obj, dict):
        for k, v in obj.items():
            out += flatten_strings(v, path+(k,))
    elif isinstance(obj, list):
        for i, v in enumerate(obj):

```

```

        out += flatten_strings(v, path+(i,))
    elif isinstance(obj, str):
        out.append((path, obj))
    return out

def load_yaml(p: Path) -> Any:
    if yaml is None:
        raise RuntimeError("PyYAML required")
    return yaml.safe_load(p.read_text(encoding="utf-8"))

def save_yaml(p: Path, data: Any) -> None:
    if yaml is None:
        raise RuntimeError("PyYAML required")
    p.write_text(yaml.safe_dump(data, allow_unicode=True, sort_keys=False), encoding="utf-8")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("yaml", type=str)
    ap.add_argument("--whitelist", type=str, required=True)
    ap.add_argument("--verify-with-music21", action="store_true")
    ap.add_argument("--autofix", action="store_true")
    ap.add_argument("--out", type=str, default=None)
    ap.add_argument("--report", type=str, default=None)
    args = ap.parse_args()

    data = load_yaml(Path(args.yaml))
    wl = load_yaml(Path(args.whitelist))
    policy = (wl.get("policy") or {}).get("unknown", "warn")

    # roots×suffix から生成
    roots = set(wl.get("roots") or [])
    suffix_groups = wl.get("suffix") or {}
    suffixes = set()
    for g in suffix_groups.values():
        for s in g.get("allow", []):
            suffixes.add(s)
    explicit = set(wl.get("explicit") or [])

    def allowed(ch: str) -> bool:
        if ch in explicit:
            return True
        # root / suffix分解 (簡易)
        m = re.match(r"^[A-G](?:[#-])?(.*)$", ch)
        if not m: return False
        root, suf = m.group(1), m.group(2)
        if root not in roots: return False
        return suf in suffixes

```

```

flat = flatten_strings(data)
failures = []
fixed = 0
verified = 0

# 直接YAMLに反映するため、オブジェクトを辿って更新
def set_in(obj: Any, path: Tuple[Union[str,int],...], value: Any):
    cur = obj
    for p in path[:-1]:
        cur = cur[p]
    cur[path[-1]] = value

for path, text in flat:
    s = text.strip()
    if not CODE_LIKE.match(s):
        continue
    norm = normalize(s)
    if norm != s:
        set_in(data, path, norm)
        fixed += 1
    if args.verify_with_m21:
        ok, err = verify_m21(norm)
        if ok:
            verified += 1
        else:
            failures.append({"path": "→".join(map(str, path)), "symbol": norm, "error": err
or "parse"})
        # whitelist check
        if not allowed(norm):
            msg = {"path": "→".join(map(str, path)), "symbol": norm, "issue": "not in
whitelist"}
            if policy == "error":
                failures.append(msg)
            elif policy == "warn":
                print("[WARN]", msg)
            else:
                pass

    if args.autofix:
        outp = Path(args.out) if args.out else
Path(args.yaml).with_name(f"fixed_{Path(args.yaml).name}")
        save_yaml(outp, data)

rep = {
    "source": args.yaml,
    "fixed": fixed,
    "verified": verified,
    "failures": failures,
}
js = json.dumps(rep, ensure_ascii=False, indent=2)
print(js)

```

```
if args.report:
    Path(args.report).parent.mkdir(parents=True, exist_ok=True)
    Path(args.report).write_text(js, encoding="utf-8")

if __name__ == "__main__":
    main()
```

4) 使い方（順序）

1) あなたのコード一覧（例： `my_chords.txt` ）を用意（1行1コード、CSV/TSVも可）。

2) ホワイトリスト生成

```
python build_whitelist_from_list.py my_chords.txt --out chord_symbol_whitelist.yaml
```

途中で **music21未受理**があればコンソールに出ます（表記修正のヒントに）。

3) YAML検証・整形

```
python validate_chordmap_plus.py chordmap.yaml
--whitelist chord_symbol_whitelist.yaml
--autofix --out fixed_chordmap.yaml
--verify-with-music21 --report out/report.json
```

- `fixed_chordmap.yaml` に正規化済みを書き出し - ホワイトリスト外は `policy.unknown` に従って警告/エラー

5) ヒント

- ルートのフラットは `-` (**B-**)，テンションのフラットは `b` (**b9**)。括弧/カンマは禁止。
- `alt` は使わず、個別列挙（`7b9#9#11b13` など）。
- `N.C.` は `explicit` に含めておく安全。
- CIでは `validate_chordmap_plus.py` をプリフックにすると常時クリーンを保てます。

以上です。 `my_chords.txt` をいただければ、このテンプレをもとに初期ホワイトリストを生成して差し戻します。